

关于二叉搜索树（Binary Search Tree，简称BST），以下是一些关键信息和程序实现要点：

一、二叉搜索树的概念

二叉搜索树是一种特殊的二叉树，它满足以下性质：

左子树性质：若左子树不为空，则左子树上所有节点的值都小于根节点的值。

右子树性质：若右子树不为空，则右子树上所有节点的值都大于根节点的值。

递归性质：左、右子树也分别为二叉搜索树。

由于二叉搜索树的这些性质，我们可以高效地进行插入、删除和查找操作，这些操作的时间复杂度在平均情况下为 $O(\log n)$ ，其中 n 是树中节点的数量。然而，在最坏情况下（例如，当树退化为链表时），这些操作的时间复杂度会退化为 $O(n)$ 。

二、二叉搜索树的程序实现

在C/C++中，二叉搜索树通常也是通过链表结构来实现的，即每个节点包含数据域和指向左右子节点的指针。以下是一个简单的二叉搜索树类的示例代码，包含了插入、查找和删除操作的基本实现：

cpp

Copy Code

```
#include <iostream>
```

```
using namespace std;
```

// 二叉搜索树节点类

```
class TreeNode {
```

```
public:
```

```
    int data; // 数据域
```

```
    TreeNode* left; // 左子节点指针
```

```
    TreeNode* right; // 右子节点指针
```

```
    TreeNode(int val) : data(val), left(nullptr),  
    right(nullptr) {}
```

```
};
```

// 二叉搜索树类

```
class BinarySearchTree {
```

```
private:
```

```
    TreeNode* root; // 根节点指针
```

// 辅助函数：插入节点（递归）

```
TreeNode* insertRec(TreeNode* node, int data) {
```

```
    if (node == nullptr) {
```

```
        return new TreeNode(data);
```

```
    }
```

```
    if (data < node->data) {
```

```
        node->left = insertRec(node->left, data);
```

```
    } else if (data > node->data) {
```

```
        node->right = insertRec(node->right, data);
```

```
    }
```

```
    return node;
}
```

// 辅助函数：查找节点（递归）

```
TreeNode* searchRec(TreeNode* root, int key) {
    if (root == nullptr || root->data == key) {
        return root;
    }
    if (root->data < key) {
        return searchRec(root->right, key);
    }
    return searchRec(root->left, key);
}
```

// 辅助函数：删除节点（递归）

```
TreeNode* deleteRec(TreeNode* root, int key) {
    if (root == nullptr) {
        return root;
    }
    if (key < root->data) {
        root->left = deleteRec(root->left, key);
    } else if (key > root->data) {
        root->right = deleteRec(root->right, key);
    } else {
        // 节点只有一个子节点或没有子节点
        if (root->left == nullptr) {
            TreeNode* temp = root->right;
            delete root;
        }
    }
}
```

```

        return temp;
    } else if (root->right == nullptr) {
        TreeNode* temp = root->left;
        delete root;
        return temp;
    }

```

// 节点有两个子节点，找到中序遍历的前驱或后继节点

```

        TreeNode* temp = minValueNode(root-
>right);
        root->data = temp->data;
        root->right = deleteRec(root->right, temp-
>data);
    }
    return root;
}

```

// 辅助函数：找到以node为根的子树中的最小值节点

```

TreeNode* minValueNode(TreeNode* node) {
    TreeNode* current = node;
    while (current->left != nullptr) {
        current = current->left;
    }
    return current;
}

```

public:

```
BinarySearchTree() : root(nullptr) {}
```

// 插入节点

```
void insert(int data) {  
    root = insertRec(root, data);  
}
```

// 查找节点

```
bool search(int key) {  
    return searchRec(root, key) != nullptr;  
}
```

// 删除节点

```
void deleteNode(int key) {  
    root = deleteRec(root, key);  
}
```

// 中序遍历（用于调试和展示树结构）

```
void inorder() {  
    inorderRec(root);  
    cout << endl;  
}
```

// 辅助函数：中序遍历（递归）

```
void inorderRec(TreeNode* root) {  
    if (root != nullptr) {  
        inorderRec(root->left);
```

```
        cout << root->data << " ";  
        inorderRec(root->right);  
    }  
}
```

```
};
```

```
int main() {
```

```
    BinarySearchTree bst;
```

```
    bst.insert(50);
```

```
    bst.insert(30);
```

```
    bst.insert(20);
```

```
    bst.insert(40);
```

```
    bst.insert(70);
```

```
    bst.insert(60);
```

```
    bst.insert(80);
```

```
    cout << "中序遍历结果：\n";
```

```
    bst.inorder();
```

```
    cout << "查找60是否存在： " << (bst.search(60) ?  
"是" : "否") << endl;
```

```
    bst.deleteNode(20);
```

```
    cout << "删除20后的中序遍历结果：\n";
```

```
    bst.inorder();
```

```
    return 0;
```

```
}
```

在这个示例中，TreeNode类表示二叉搜索树的节点，包含数据域data和左右子节点指针left、right。BinarySearchTree类表示二叉搜索树，包含了根节点指针root以及插入、查找、删除和中序遍历的辅助函数。在main函数中，我们创建了一个二叉搜索树，并进行了插入、查找和删除操作，最后通过中序遍历展示了树的结构。